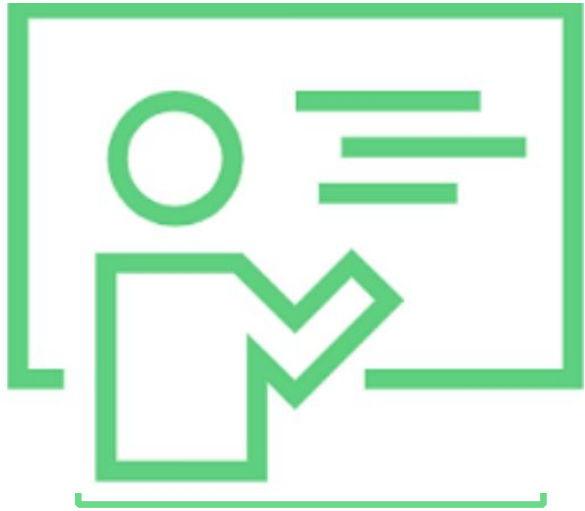




Nätverksprestanda och systematisk felsökning

Joakim Englund, Systemmentor AB

Repetition från dag 9

- Loggar beskriver enskilda händelser
- Mätetal visar mängd, frekvens och svarstid
- Nätverksspår visar vad som passerar gränssnittet
- Korrelations-ID binder ihop samma förlopp

Dagens mål

- Mäta en reproducerbar baslinje
- Förklara latency, jitter, genomströmning och felandel
- Skilja anslutningsfel från applikationsfel
- Pröva en hypotes med minsta relevanta bevis
- Motivera en åtgärd med före- och eftermätning

Fråga före verktyg

Observation → Fråga → Hypotes → Lösning → Test → Bevis → Slutsats

- "Det är nätverket" är inte en tillräckligt specifik hypotes för att vara särskilt användbar. Hypotesen ska kunna bevisas eller motbevisas och helst peka på specifika delar i systemet som vi sen kan testa.
- "Fördröjningen uppstår efter att servern tagit emot begäran" är bättre. Den kräver visserligen att vi har tillgång till servern, men det har vi nu och generellt sett är vi ju med mer eller mindre i hela kedjan från sensorenheter till molntjänster så det ska gå att få tillgång till vid felsökning.

Fyra prestandabegrepp

Begrepp	Fråga
Latency	Hur lång tid tar ett förlopp?
Jitter	Hur mycket varierar tiden?
Genomströmning	Hur många lyckas per tidsenhet?
Felandel	Hur stor andel misslyckas?

Det kan även vara relevant att titta på 'paketförlust', men när vi kör med TCP så kommer ju förlorade paket att sändas om och därmed täckas upp av 'genomströmning'.

Definiera mätningen

- Startpunkt och slutpunkt
- Enhet
- Antal observationer
- Tidsintervall
- Testdata och konfiguration

Baslinje före förändring

Samma scenario + känd konfiguration + flera observationer = baslinje

- Spara både normal variation och avvikelse
- Optimera inte innan flaskhalsen är lokaliserad

Medelvärde r cker inte

10, 11, 12, 13, 500 ms

Medel: 109,2 ms

Medianen/P50 (50:e percentilen) + max/P95 visar beteendet b ttre

- Visa antal m tningar
- Visa minst medel och ett toppm tt
- 5 m tv rden ger v ldigt grov  verblick. Detta var bara ett exempel - sikta p  lite fler (10-20  tminstone)

Genomströmning behöver felandel

genomströmning = lyckade operationer / tid

felandel = misslyckade operationer / alla försök

- Hög utskickstakt ger inte automatiskt hög nyttig genomströmning
- Tappat arbete måste synas

Lager och gränser

Länk → IP/DNS → Port/TCP → TLS → HTTP/MQTT → Data → Tjänst

- Varje gräns har egna observationer
- Börja där bevisen pekar

Tre olika fel

<i>Symptom</i>	<i>Trolig nivå</i>
Connection refused	Ingen lyssnar på adress/port
HTTP 500	Tjänsten svarar men arbetet misslyckas
HTTP 400	Begäran når tjänsten men datan avvisas

Ping är inte test av tjänst

- Ping kan visa IP-nåbarhet
- ICMP kan vara blockerat trots fungerande tjänst
- Svar på ping visar inte att port 8091 är öppen
- Testa så nära det verkliga protokollet som möjligt

Klienttid och servertid

Klientens totaltid:

DNS + anslutning + överföring + serverarbete + svar

Servers arbetstid:

serverarbete

Wireshark (och tcpdump)

Capture filter: tcp port 8091

Display filter: tcp.port == 8091

- SYN utan svar pekar mot anslutningsvägen
- Etablerad anslutning bevisar inte korrekt applikationsdata
- Omsändningar är en observation, inte automatiskt rotorsaken

Ändra en sak i taget

Baslinje → En ändring → Samma test → Jämför → Återställ

- Flera samtidiga ändringar förstör orsakskedjan
- Spara kommandot och konfigurationen
- En omstart kan maskera felet. Dokumentera om omstarten ingår i åtgärden

Demo

- Terminal 1: Lokal gateway/server med logg och behandlingstid
- Terminal 2: Simulerad sensor och mätklient
- Terminal 3/app, valfri: Passiv nätverksobservatör (Wireshark/tcpdump)

Demo: normalfall

Jag kör med python, men C++-programmet tar samma parametrar

```
python server.py
```

```
python client.py --count 20
```

- Sparar t.ex. min, medel, p95, max och felandel
- Följer ett `request\_id` i klient och server

Demo: kontrollerad fördröjning

```
python server.py --delay-ms 250  
python client.py --count 20
```

- Ökar klienttid och `processing\_ms` tillsammans?
- Vad talar det för och emot?

Demo: anslutningsfel

```
python client.py --port 8092 --count 3
```

- Ingen HTTP-status
- Ingen motsvarande serverlogg
- Klienten rapporterar anslutningsfel

En motiverad åtgärd

- Kopplas till en testad hypotes
- Är så liten som möjligt
- Mäts med samma scenario före och efter
- Har dokumenterad bieffekt eller begränsning
- Kan återställas

Laborationens definition av klar

- Dokumenterad baslinje
- Kontrollerat fel och testbar hypotes
- Helst två beviskällor
- Lokaliserad felgräns
- Före/efter eller motiverad rekommendation
- Verifierad återställning